SECURITY AUDIT

Pancakeswap

Infinity Hook

FINAL REPORT

July 2026

BAILSEC.IO

Disclaimer:

BailSec was engaged by the client to perform a time-boxed technical security assessment of the target identified in this report. The assessment applies only to the scope, version, commit, deployment, materials and information identified in the report. Its findings are point-in-time and not exhaustive. Changes made after the assessment—including remediation changes—were not assessed unless expressly stated otherwise and may affect the applicability of the report's conclusions.

No security assessment can identify every vulnerability or guarantee that a system is secure, error-free or protected against attack or loss. Only the components, assumptions, threat models and risk categories expressly identified as in scope were assessed. The report should be read in full, including its scope, exclusions, unresolved findings and remediation status. The client remains responsible for design, testing, remediation, deployment, operation, monitoring and incident response.

This report is a technical assessment, not a certification, attestation, investment recommendation, or legal, financial, regulatory or tax advice. It does not constitute an endorsement of the client, the assessed system or any related project.

This report was prepared for the client under a separate engagement agreement. If published, its public availability is for transparency and informational purposes only. To the fullest extent permitted by applicable law, publication does not create an auditor-client relationship, duty of care or right of reliance in favor of any third party. Other readers should conduct their own diligence and obtain appropriate professional advice.

Only the complete, unmodified report as issued by BailSec is authoritative. The signed engagement agreement exclusively governs the contractual rights and obligations between BailSec and the client, including warranties, remedies, liability limitations, indemnities and dispute resolution. Nothing in this notice excludes liability that cannot lawfully be excluded.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Pancakeswap – Infinity Hook – Audit Report
Website	pancakeswap.finance
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/pancakeswap/infinity-shared-inventory-hook/tree/a1ad99937cd2a74c8ff6218b08e349f2c4a0640c/src/cl
Resolution 1	https://github.com/pancakeswap/infinity-shared-inventory-hook/tree/95e8ede66dd5d062a82abd0cb7171c8cbbda989d

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	3			3		
Low	16	2		14		
Informational	8	3	1	4		
Governance	1			1		
Total	28	5	1	22		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

2. Detection

CLSharedInventoryHook

The CLSharedInventoryHook contract is a PancakeSwap Infinity (CLAMM) hook that market-makes pools out of a shared inventory of tokens held directly by the hook, injecting liquidity just-in-time in beforeSwap and withdrawing it in afterSwap within the same transaction so capital is never idle-locked per pool. Any token pair may bind to the hook since there is no designated hub token; the owner sets a fixed tick range and liquidity amount per pool, and inventory is simply the hook's live balance per currency with no separate bookkeeping, so a token shared across several pools is drawn from one common balance. Insufficient inventory or a breached reserve floor reverts the swap, and all swap-path token movement is settled against the vault's flash-accounting inside the same lock the swap runs in.

Appendix: Deferred just-in-time removal

Infinity only books a swap's incoming token to the pool manager's reservesOfApp after afterSwap returns, but the just-in-time position being removed in afterSwap already holds that just-swapped-in token, so removing it inline would drive reservesOfApp negative and underflow. When that happens, afterSwap catches the failure, pays only the still-owed injection delta, leaves the position live, and snapshots it in _pendingRemoval together with its pool id. The very next beforeSwap call on any pool unwinds that parked position first, since by then the earlier swap has fully settled and the burn nets out safely; at most one deferred removal can exist at any time.

Appendix: Reserve floor enforcement on the deferred path

_enforceFloors checks minSharedReserve for both currencies after each swap, but the check differs by removal path. On the inline path, the live balance already reflects the unwound position. On the deferred path, the position is still parked in the pool, so _enforceFloors adds its principal at the current price (via _requiredAmounts) to the live balance before comparing against the floor, deliberately excluding this swap's uncollected LP fees as a conservative under-count.

Privileged Functions

- setPoolConfig
- setUninitializePoolConfig
- setPoolConfigs
- setUninitializePoolConfigs
- setMinSharedReserve
- setJitPaused
- setSwapPausedFor
- setRestrictedPoolCreation
- setRequireEnabledConfig
- setDisableDeferredRemoval
- setInitialSqrtPrice
- setPoolCreator
- setWithdrawRecipient
- withdraw
- emergencyRemoveLiquidity

Invariants

- At most one deferred removal (_pendingRemoval) exists across all pools at any time.
- Each currency moves at most once per swap, since the injection delta and removal delta are always netted and settled together.
- A withdrawal or swap settlement can never push a currency's live balance below its minSharedReserve floor.
- A pool can only be initialized at a permissionless price if it exactly matches the owner-approved permissionlessInitSqrtPriceX96.
- Timed swap halts never exceed MAX_SWAP_PAUSE and are spaced at least SWAP_PAUSE_COOLDOWN apart.

Issue_01	Several owner-controlled inputs lack bounds, and role/privilege separation has structural gaps
Severity	Governance
Description	The hook has strong economic guardrails (reserve floors, anti-front-run caps on config, solvency previews) but weak bounds validation on owner-supplied inputs, and the owner contract's role separation has several structural gaps. These are collected here as governance-level observations. **A. Unbounded or ungated values** - **cfg.liquidity has no upper bound.** _applyPoolConfig only rejects L == 0. The owner can set L to any uint128. If the resulting required amounts exceed the hook's balance, _settleNet reverts and every swap on that pool reverts. Example: the hook holds 100 USDC and the owner sets an L needing 1,000 USDC. The pool is dead until someone reconfigures or funds it. The checkSolvency flag is opt-in (a bool the caller passes), so the owner can skip the one check that would catch this. - **Tick range is never checked against the pool's tick spacing.** The code acknowledges this at line 803: alignment "isn't checked here; modifyLiquidity enforces alignment at swap time." So setPoolConfig succeeds and emits PoolConfigured for a range like [-100, 100] on a tickSpacing == 60 pool. There is no try/catch around the mint in _beforeSwap (line 336). Result: the config looks applied, and every swap on that pool reverts from then on. This is a silent misconfiguration with no on-chain feedback. - **No limit on range width.** [MIN_TICK, MAX_TICK] is accepted, as is a single-spacing-wide range. A near-zero-width range parked just off the current price makes the JIT position effectively one-sided: the hook buys only one leg and accumulates inventory imbalance every swap.

- **maxRequired0 / maxRequired1 can be type(uint256).max.** Passing max opts out of the anti-front-run guard entirely (line 817). The pre-creation setUninitializePoolConfig path hardcodes max (lines 667-668), so no cap is even possible there.

- **setInitialSqrtPrice accepts any valid price and never expires.** It is bounded only by MIN_SQRT_RATIO / MAX_SQRT_RATIO (line 985). If the owner approves a price and the market moves, anyone can later permissionlessly initialize that pool at the now-stale price. The contract's own documentation (lines 134-139) acknowledges the pre-funding requirement, but nothing enforces it.

B. emergencyRemoveLiquidity lacks _isActive guard, enabling nested-lock re-entry via token callbacks

emergencyRemoveLiquidity acquires a fresh vault.lock and, unlike withdraw, carries no _isActive() check. Its callback settles through token transfers while the in-swap flag is not raised. If the vault permits nested locks (or a token transfer callback inside the settlement can open a swap while the emergency lock is held), the hook's emergency path can interact with the swap path mid-settlement. The nonReentrant modifier guards re-entry into this contract's external functions but does not cover a currency's transfer callback firing inside _settleNet and reaching the swap path where _isActive() is false. This requires a trusted owner to permit a callback-capable token, making it a governance concern rather than an external exploit vector.

C. CONFIG_ROLE can activate stale price approvals without CREATOR_ROLE

The permissionlessInitSqrtPriceX96 (price pin) and _poolConfig are independent storage domains. setUninitializePoolConfig rewrites the pool config without clearing or rebinding the price pin. A CONFIG_ROLE holder can therefore re-size the liquidity L that an already-approved,

anyone-can-open pool will inject, inheriting the authorization power the design reserved for CREATOR_ROLE without holding it. The atomic owner route (CLSharedInventoryHookOwner) requires both CONFIG_ROLE and CREATOR_ROLE, but the standalone config setter does not enforce this coupling.

D. setAdmin grants DEFAULT_ADMIN_ROLE without a revoke primitive

setAdmin on the owner contract only adds a DEFAULT_ADMIN_ROLE holder. There is no removeAdmin or owner-callable _revokeRole in the owner contract. The inherited AccessControl.revokeRole is itself gated by DEFAULT_ADMIN_ROLE, so an admin can only be removed by another admin, never by the owner recovery key. Once an admin address is compromised, the owner cannot cleanly evict it and must resort to a full owner-contract migration. It is advised to expose a removeAdmin / _revokeRole function callable by the owner for recovery.

E. DEFAULT_ADMIN_ROLE self-administers, collapsing separation of duties

The owner contract never calls _setRoleAdmin for any role. Under stock OpenZeppelin AccessControl, every role's admin defaults to DEFAULT_ADMIN_ROLE, including DEFAULT_ADMIN_ROLE itself. The documented separation of duties (e.g. recipient-allowlist management kept in DEFAULT_ADMIN_ROLE, separate from WITHDRAW_ROLE which decides when to withdraw) is unenforceable: a single admin key can grantRole(WITHDRAW_ROLE, self), allowlist its own address via setWithdrawRecipient, zero every reserve floor via a self-granted RESERVE_ROLE, and withdraw the entire inventory in one transaction.

F. Admin can self-grant WITHDRAW_ROLE and drain inventory

setWithdrawRecipient is gated by DEFAULT_ADMIN_ROLE to keep the "where funds may go" decision separate from the "when to withdraw"

	decision (WITHDRAW_ROLE). However, because DEFAULT_ADMIN_ROLE administers all roles, a single admin key can grant itself WITHDRAW_ROLE, allowlist its own address, and withdraw the full non-floor inventory. This collapses the stated two-key requirement into a single key. The ceiling is the withdrawable inventory the admin already controls, so it is a governance-centralization observation rather than an external exploit.
Comments / Resolution	Acknowledged with the following notes: Tick ranges are now validated in _applyPoolConfig before config is set. For remaining items, PancakeSwap will consider adding further restrictions in future iterations of the owner contract.

Issue_02	Deferred JIT removal can cause multihop swaps to revert
Severity	**Medium**
Description	The router executes every swap in a multihop route before settling the user's initial input. If the first pool cannot remove its JIT liquidity immediately, the hook records it as a pending removal. When the router starts the next hop, that pool's beforeSwap calls _flushPendingRemoval. Flushing the previous position returns tokens to the hook through vault.take. However, the router has not yet transferred the user's initial input into the Vault. Therefore, the Vault's accounting may show that the tokens are available while its actual token balance is still insufficient to make the transfer. The vault.take call then reverts, causing the complete multihop transaction to fail. As a result, ordinary exact-input and exact-output routes through multiple pools using this hook can unexpectedly revert.
Recommendations	Do not flush a pending JIT position during a later hop before the router has completed settlement.
Comments / Resolution	Acknowledged.

Issue_03	Arbitrage possible if preset price is different from current market price at initialization
Severity	Medium
Description	Owner can set an initial price via the setInitialSqrtPrice function for a particular pool. There is no expiry for this setting, so it can be used to initialize the pool anytime. Any user can then come in and create and initialize that pool at any time. If the current market price is different from the preset initial sqrt price, they can even profit by arbitraging against the JIT liquidity in the contract, depleting the hook contract, which will bear the loss.
Recommendations	Consider adding a time-bound for the initial price setting, so that stale initial prices cannot be used to create new pools.
Comments / Resolution	Acknowledged.

Issue_04	Deferral safety mode deniable indefinitely
Severity	Medium
Description	setDisableDeferredRemoval(true) is blocked whenever a pending-removal record exists. Any swapper can trigger a deferred removal if the conditions are right, and the admin cannot guarantee a window where this pending record will be empty. So it can be very difficult to disable deferred removals.
Recommendations	Consider flushing pending removals in the setter function itself, to prevent the revert.
Comments / Resolution	Acknowledged.

Issue_05	Anyone can trigger a deferred removal
Severity	**Low**
Description	Normally, a deferred removal is triggered only when reservesOfApp underflows during liquidity extraction after the swap, which happens because the vault accounting updates after the hook. Since reservesOfApp is shared between all the pools in the vault, users can remove liquidity from some other pool in the vault which uses that token to reduce reservesOfApp and trigger the deferred removal branch of the contract. Thus the deferred branch can be manually triggered by the end user.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_06	Creation front-run forces manipulable config route
Severity	**Low**
Description	The price-safe pre-creation config route reverts if the pool already exists. An attacker who front-runs pool creation permanently closes that route for the pool, forcing the operator onto the PoolId route that sizes against the manipulable live price, exactly the surface the pre-creation route was meant to avoid.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_07	JIT liquidity can be stolen by sandwiching pause switches
Severity	**Low**
Description	If setJitPaused is called by an admin to enable JIT in a pool, the liquidity availability in the pool suddenly changes. However, there are no guards here making sure that this switch is flipped at a realistic price. Thus, an attacker can frontrun this call to manipulate the price of the pool and profit from the JIT liquidity. Say the pool has tokens A and B. An attacker can swap A tokens into BB tokens right before setJitPaused is called to enable JIT, and then sell back the B tokens once JIT is enabled. Since the selling leg has more liquidity depth, it will yield more A tokens than the initial purchase, netting a profit for the caller, at the cost of the liquidity in the hook. This is because neither the admin function nor the hook checks the current price before adding the JIT liquidity to the pool. This issue is marked Low severity as it's acknowledged in the Natspec docs.
Recommendations	Consider acknowledging this issue, or adding a slippage guard in setJitPaused similar to config change functions.
Comments / Resolution	Acknowledged.

Issue_08	A single unflushable pending removal halts all swaps
Severity	**Low**
Description	There is a single global pending-removal slot, unwound at the top of every swap with no try/catch. If that one parked position ever fails to burn/settle, e.g., its token becomes non-transferable after deferral — the flush reverts, and because it runs before every swap on every pool, all hook-bound pools stop swapping until the owner intervenes.
Recommendations	Consider acknowledging this issue and documenting this behaviour.
Comments / Resolution	Acknowledged.

Issue_09	beforeInitialize does not enforce the same sqrtPrice as used when setting the config
Severity	**Low**
Description	In setUninitializePoolConfig, the caller specifies a sqrtPriceX96 which is then used for the checkSolvency check. This initializes the config for the pool, aimed at an initial price of sqrtPriceX96. But the beforeInitialize hook never ensures that the initial price equals the same sqrtPriceX96 used during setup. Thus, a config can be set up using a certain price and the pool can be initialized at a different price, leading to an incorrect solvency check, which can lead to reverts during the enforceFloors checks.
Recommendations	Consider enforcing the same price during config setup and pool initialization.
Comments / Resolution	Acknowledged.

Issue_10	tx.origin authorization on creation gate
Severity	**Low**
Description	The creator whitelist is checked against tx.origin, so any contract a whitelisted EOA transacts through inherits that EOA's pool-creation authority, including the choice of initial price. A malicious or compromised intermediary can create hook-bound pools on the EOA's behalf.
Recommendations	Consider acknowledging this issue, or restricting creations in some other manner.
Comments / Resolution	Acknowledged.

Issue_11	External LP withdrawal can force gross JIT settlement and block swaps
Severity	**Low**
Description	The hook adds JIT liquidity in beforeSwap without immediately paying the mint cost. If the position can be removed successfully in afterSwap, the addition and removal deltas are combined: *_collectRemoval(* *id,* *key,* *injectDelta + removeDelta,* *feeDelta* *);* This means the hook normally pays only the small net amount produced by the swap instead of the full cost of minting the JIT position. Successful inline removal depends on the PoolManager having sufficient reservesOfApp. Resident liquidity can provide this reserve buffer. However, an external LP can withdraw its resident liquidity at any time. After the buffer is removed, the next swap still adds the complete JIT position in beforeSwap. Its removal may then underflow the available reservesOfApp and enter the deferred-removal branch: *catch {* *if (_controls.disableDeferredRemoval) revert RemovalDeferralDisabled();* *_settleNet(key, injectDelta);* *_enforceFloors(id, key, cfg, true);* *_pendingRemoval = PendingRemoval({* *id: id,* *cfg: cfg* *});* *}*

	Because there is no removal delta to combine with the injection in this branch, _settleNet must pay the full gross mint cost represented by injectDelta. The settlement function reverts if the hook does not hold the complete amount: *if (net0 < 0) {* *uint256 owe0 = uint256(uint128(-net0));* *uint256 bal0 = _balanceOf(key.currency0);* *if (bal0 < owe0) {* *revert InsufficientInventoryToSettle(* *key.toId(),* *key.currency0,* *bal0,* *owe0* *);* *}* *key.currency0.settle(vault, address(this), owe0, false);* *}* A hook can therefore have sufficient inventory to cover the normal net swap movement but insufficient inventory to pay the full gross JIT mint cost. Such a pool works while the external resident buffer exists, but begins reverting after the external LP withdraws it.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_12	External LPs can exhaust JIT boundary capacity and block swaps
Severity	**Low**
Description	The pool limits the total liquidity that can exist in a single tick. The hook adds its configured JIT liquidity before every swap without checking whether enough capacity remains at both boundary ticks. An external LP can add liquidity using one of the hook's boundary ticks until the remaining capacity is smaller than the configured JIT amount. The hook's next liquidity injection then exceeds the per-tick limit and reverts, causing the complete swap to fail. Swaps can continue reverting while the external position remains and JIT is enabled.
Recommendations	Check the remaining capacity of both boundary ticks before injecting JIT liquidity. If either tick lacks sufficient capacity, skip JIT injection and allow the swap to continue using resident liquidity.
Comments / Resolution	Acknowledged.

Issue_13	withdraw does not flush the pending deferred removal before checking balances
Severity	Low
Description	The _flushPendingRemoval function, which settles a deferred JIT position and returns its tokens to the hook's balance, is only called at the top of _beforeSwap. The withdraw function does not trigger a flush before checking the hook's live balance. When a deferred removal is pending, the tokens locked in that position (both principal and accrued fees) are not included in _balanceOf(currency), because the position is still live in the pool manager. The _withdraw function then compares the requested amount against this undercounted balance: *uint256 bal = _balanceOf(currency);* *if (amount > bal || bal - amount < floor) {* *emit WithdrawSkipped(currency, bal, floor, amount);* *return;* If the pending position holds enough tokens to satisfy the withdrawal, the check still fails, and the entry is silently skipped with a WithdrawSkipped event. The owner receives no revert and no funds; the withdrawal simply does not execute. This is particularly likely right after a swap that deferred its removal: the hook's live balance is at its lowest (the position's tokens have not been returned yet), and any withdrawal attempted before the next swap will see a deflated balance.
Recommendations	Consider flushing the pending deferred removal at the start of withdraw so that the balance check reflects the hook's full available inventory.
Comments / Resolution	Fixed: removePendingAndWithdraw removes pending liquidity before withdrawing.

Issue_14	Unconditional +1 in _requiredAmountsAt can lead to issues
Severity	**Low**
Description	The _requiredAmountsAt helper adds +1 to both amount0 and amount1 after calling getAmountsForLiquidity, to compensate for the pool's round-up on mint: *amount0 += 1;* *amount1 += 1;* The issue is that this is done for all calls, irrespective of mint/burn direction, even when the market price is outside of the config bounds. This leads to a number of issues: 1. **checkSolvency rejects valid single-sided configs**. When the price is above the range, amount1 is genuinely 0. The +1 inflates it to 1, so the solvency check at line 826 evaluates 1 > 0 and reverts with InsufficientInventory for a currency the pool will never draw. The owner must either dust the hook with 1 wei of an unneeded token or disable checkSolvency entirely. 2. **maxRequired caps reject correct inputs.** A caller who correctly passes maxRequired1 = 0 (expecting zero cost on that side) triggers RequiredExceedsMax(required0, maxRequired0, 1, 0) at line 816. The natural way to express "I expect zero here" is the one value that fails. 3. **_enforceFloors:_requiredAmounts rounds in the wrong direction**. _requiredAmounts is meant to calculate the funds composition of the deferred withdrawal. However, instead of rounding down, it rounds up due to the +1 addition, and thus overestimates the pending withdrawal by 1 wei.
Recommendations	Consider rounding up only for mints, and ignoring the rounding if the market price is outside of the tick range..
Comments / Resolution	Acknowledged.

Issue_15	Race condition between setInitialSqrtPrice revocation and permissionless pool initialization
Severity	Low
Description	The permissionlessInitSqrtPriceX96 mechanism allows the owner to open a specific pool to permissionless creation at a pinned price. Revoking that approval requires a separate setInitialSqrtPrice(id, 0) call, which sets the mapping back to zero and re-locks creation behind the isPoolCreator whitelist. Because restrictedPoolCreation is a global flag (not per-pool), any pool with a non-zero permissionlessInitSqrtPriceX96 is open to anyone while the flag is on. If the owner sets price A for a pool and later decides to revoke it, there is a window between the two transactions where a non-whitelisted caller can front-run the revocation and initialize the pool at price A. The owner cannot atomically set and revoke the approved price. Once the setInitialSqrtPrice(id, A) transaction confirms, the pool is immediately initializable by anyone at that exact price, and the revocation is a separate transaction that can be front-run.
Recommendations	Consider making the pool-creation restriction a per-pool setting rather than a global flag, so the owner can gate individual pools without exposing a revocation window. Alternatively, consider allowing the owner to batch setInitialSqrtPrice and pool initialization atomically, so the approval never exists in a publicly exploitable state.
Comments / Resolution	Acknowledged.

Issue_16	Out-of-range swaps still mint and burn JIT liquidity, bleeding 1 wei per swap and risking revert
Severity	Low
Description	When the pool's price is outside the configured JIT tick range, _beforeSwap still mints the full L into the pool. Because the price is out of range, only one leg is touched, and the swap never crosses the position. The _afterSwap burn prices off the same out-of-range branch, returning the same single token. Due to rounding asymmetry between mint (rounds up) and burn (rounds down), the net injectDelta + removeDelta is (-1 wei, 0) or (0, -1 wei) rather than exactly zero. _settleNet pays this 1 wei from the hook's inventory. The JIT position earns no fees because the swap never entered its range; the hook pays a full mint-burn round trip for nothing and loses 1 wei of inventory per swap. This is the dust bleed the NatSpec at lines 861-864 already documents. Beyond the slow drain, the 1 wei settle can also cause a revert: if the hook holds zero balance of the out-of-range leg's currency, _settleNet fails with InsufficientInventoryToSettle. This means a swap that never touches the JIT range reverts when it did not need JIT liquidity at all.
Recommendations	Consider skipping the JIT injection in _beforeSwap when the pool's current price is entirely outside the configured tick range, since the position cannot participate in the swap and only costs rounding dust.
Comments / Resolution	Acknowledged.

Issue_17	disableDeferredRemoval is global, forcing all pools onto resident-buffer mode when only one pool is problematic
Severity	Low
Description	The disableDeferredRemoval flag is a single global boolean. When it is set to true, _afterSwap reverts with RemovalDeferralDisabled on any pool whose removal would underflow, and _beforeSwap skips _flushPendingRemoval entirely. Every pool bound to the hook must then carry a resident liquidity buffer large enough to settle removals inline, or its swaps revert. In practice, this flag is toggled because of a single pool whose token has weird traits (blocklist, pause, freeze) that brick the flush. The owner disables deferral globally so the stuck flush does not halt swaps on other pools. However, pools whose tokens have no such traits lose the deferral mechanism unnecessarily; they are forced into the stricter inline-settlement mode and require a standing buffer they did not need before. This is disproportionate: one problematic pool imposes operational overhead on every other pool served by the hook.
Recommendations	Consider making the deferred-removal toggle per-pool (e.g. a mapping(PoolId => bool)) so the owner can disable deferral only for pools with problematic tokens, while allowing all other pools to continue using the deferral path without requiring a resident buffer.
Comments / Resolution	Acknowledged.

Issue_18	Emergency removal can clear a pending record without fully removing its liquidity
Severity	Low

Description	When emergencyRemoveLiquidity is called for a pool with a pending JIT removal, the function deletes the complete _pendingRemoval record after checking only the pool ID:
	
	`if (`
	`  _pendingRemoval.cfg.liquidity != 0`
	`  && PoolId.unwrap(_pendingRemoval.id) == PoolId.unwrap(key.toId())`
	`) {`
	`  delete _pendingRemoval;`
	`}`
	
	The function does not verify that the supplied ticks and liquidity match the pending position before deleting the record.
	
	If the owner removes only part of the pending liquidity, the removal can succeed, and the deletion remains effective. The unremoved liquidity stays live in the pool, but the hook no longer tracks it as pending. Therefore, later swaps will not automatically flush it.
	
	If the pool remains enabled, fresh JIT liquidity may be added on top of the remaining position, temporarily exposing more liquidity than configured. Recovery requires the owner to identify and remove the remaining position manually.

Recommendations	Only clear _pendingRemoval automatically when the supplied pool, ticks, and liquidity exactly match the pending position. For partial removals, reduce the recorded pending liquidity by the amount removed. Keep the existing zero-liquidity path as an explicit abandonment action.
Comments / Resolution	Acknowledged.

Issue_19	Deferred liquidity can cause a reserve-floor increase to be silently skipped
Severity	Low
Description	An attacker can front-run a reserve-floor increase with a swap that parks hook inventory in a deferred JIT position. setMinSharedReserve checks only the hook's directly held balance, so it emits MinSharedReserveSkipped and leaves the old floor active. The attacker can then back-run with another swap. That swap flushes the pending position but still executes under the old lower floor before the owner notices and retries.
Recommendations	Revert when a requested floor cannot be installed instead of silently skipping it, or reject floor updates while a pending removal exists.
Comments / Resolution	Fixed: removePendingAndSetMinSharedReserve() atomically removes pending liquidity before applying the new reserve floor.

Issue_20	Per-currency minSharedReserve floor causes cross-pool contagion during depeg events
Severity	Low
Description	The minSharedReserve mapping is keyed by currency, not by pool. A single floor value applies across every pool that uses that token, because the hook holds one shared balance per currency. When multiple pools share a token, a depeg in one pool can trip the floor and freeze swaps in unrelated pools. Consider two pools on the same hook: - Pool A: USDC / USDT - Pool B: USDT / DAI Both pools draw from the same USDT balance and are governed by the same USDT floor (e.g. 100,000 USDT). If USDC depegs, panicked swappers in Pool A sell USDC for USDT, draining the hook's USDT balance down to the floor. Once _enforceFloors detects bal1 < min1, it reverts with MinSharedReserveBreached for any swap that would further reduce USDT. Pool B is now blocked from handing out USDT, even though DAI never moved and the USDT/DAI pair is unaffected by the USDC depeg. The freeze is one-directional: swaps that add USDT back to the hook still succeed and can lift the breach. But during a depeg, the dominant flow is away from the depegging token, so the directional block persists until the panic subsides or the owner lowers the floor.
Recommendations	Consider supporting per-pool reserve floors (e.g. keyed by PoolId and currency) so that a depeg-driven drain in one pool does not trip the circuit breaker for unrelated pools. If per-pool floors are too costly, consider documenting that pools sharing a token are subject to contagion and that the only mitigation is to avoid token overlap across pools.
Comments / Resolution	Acknowledged.

Issue_21	setMinSharedReserve ignores pending-removal capital conservatively
Severity	**Informational**
Description	The reserve setter only checks current wallet balance: *function _setMinSharedReserve(Currency currency, uint256 amount) private {* *uint256 bal = _balanceOf(currency);* *if (amount > bal) {* *emit MinSharedReserveSkipped(currency, bal, amount);* *return;* *}* *minSharedReserve[currency] = amount;* *emit MinSharedReserveSet(currency, amount);* If a deferred removal holds additional capital in-pool, this won't count it. This is conservative. It can block an operator from setting a floor that would be valid after the next flush.
Recommendations	Consider accounting for the pending capital as well.
Comments / Resolution	Fixed: removePendingAndSetMinSharedReserve() now removes pending liquidity before setting reserve floors.

Issue_22	Code comments incorrectly state tx.origin is always an EOA post-EIP-7702
Severity	Informational
Description	The code comments claim that the pool-creator whitelist is "keyed by the ORIGINATING account, which is always an EOA" and that "only EOA creators are supported: a contract / multisig cannot be a whitelisted creator." Since the Pectra upgrade (EIP-7702), these statements are no longer accurate. A whitelisted address can delegate to arbitrary contract code via EIP-7702, making it simultaneously tx.origin and a programmable smart account. The whitelist gate itself is not bypassed by EIP-7702: tx.origin still equals the account that signed the outer transaction, so a non-whitelisted attacker cannot forge a whitelisted origin. However, a whitelisted account with 7702 delegation is effectively a "contract / multisig creator" wearing an EOA address, which contradicts the stated design assumption. The comments are misleading for anyone reasoning about the security model post-Pectra.
Recommendations	Consider updating the code comments to acknowledge that a whitelisted account may be a 7702-delegated smart account, and that the security reasoning should not rely on the assumption that tx.origin is always a plain EOA.
Comments / Resolution	Fixed. The comments were updated to account for EIP-7702 delegated smart accounts.

Issue_23	setJitPaused does not enforce idempotency
Severity	Informational
Description	The setJitPaused function allows setting jitPaused to true when it is already true, and false when it is already false. Based on the code comments, pausing should only be possible when unpaused, and unpausing when paused. Without this check, redundant state writes waste gas and emit misleading events that suggest a state transition occurred when nothing changed.
Recommendations	Consider reverting if the new value matches the current state, so pause/unpause calls always represent an actual state transition.
Comments / Resolution	Fixed: The owner wrapper now reverts when the requested pause state is already active.

Issue_24	Global swapPausedUntil may be insufficient for single-pool incidents
Severity	Informational
Description	The swapPausedUntil mechanism is global: it pauses swaps across all pools bound to the hook for up to MAX_SWAP_PAUSE (10 minutes). When an incident affects only one pool, the owner must freeze every pool to pause the affected one. Ten minutes may also be insufficient to respond to complex incidents, especially if the owner needs to coordinate with external parties or wait for governance actions.
Recommendations	Consider supporting a per-pool swap pause mapping (e.g. mapping(PoolId => uint256) poolSwapPausedUntil) so the owner can freeze individual pools without affecting others, and consider making MAX_SWAP_PAUSE configurable within a bounded range.
Comments / Resolution	Acknowledged.

Issue_25	_flushPendingRemoval in _beforeSwap can brick non-JIT swaps unnecessarily
Severity	Informational
Description	In _beforeSwap, the pending removal flush runs before the checks for jitPaused, enabled, and liquidity == 0: *if (!ctrl.disableDeferredRemoval) _flushPendingRemoval();* If _flushPendingRemoval reverts (e.g. because reservesOfApp still cannot support the removal), all swaps on every pool are bricked until the owner clears the pending position via emergencyRemoveLiquidity. This is excessive: pools where JIT is paused, disabled, or configured with zero liquidity would otherwise swap normally on resident liquidity. They do not need the flush to succeed. Since disableDeferredRemoval requires the owner to clear any pending position before enabling it, and the flush is all-or-nothing, moving the flush after the JIT-skip checks would allow non-JIT swaps to proceed unblocked while the owner resolves the stuck pending position.
Recommendations	Consider moving _flushPendingRemoval to after the jitPaused / enabled / liquidity == 0 checks, so that pools not using JIT injection are not blocked by a stuck deferred removal.
Comments / Resolution	Acknowledged.

Issue_26	Pool configurations may be stored even though Core rejects them during swaps
Severity	Informational
Description	_applyPoolConfig validates global tick bounds but does not verify that the ticks align with the pool's tick spacing. The configuration can therefore be stored successfully, while the first JIT injection reverts inside Core. Swaps remain unavailable until the owner corrects the configuration. Configurations producing token deltas outside Core's int128 accounting range may fail similarly. The pre-initialization PoolKey validation also accepts nonzero unused high bits in key.parameters. Because PoolId hashes the complete parameters word, the configuration is stored under a different PoolId, while Core later rejects that key during initialization through checkUnusedBitsAllZero(). This can leave an unusable or orphaned preconfiguration.
Recommendations	Validate tick alignment and ensure the calculated mint amounts fit Core's signed accounting range before storing an enabled configuration. Additionally, validate that all unused high bits in key.parameters are zero by applying Core's checkUnusedBitsAllZero() requirement during pre-initialization configuration.
Comments / Resolution	Partially fixed: Tick-spacing alignment is now validated, but unused parameter bits and signed amount limits are still not fully checked.

Issue_27	_removePending has no slippage check
Severity	**Informational**
Description	The _removePending function removes liquidity from the pool, but passes in 0 for minimum amounts for A and B. If liquidity is removed when the pool is at a bad price, the hook can suffer losses in dollar terms.
Recommendations	Consider passing in minimum amounts for the liquidity removal.
Comments / Resolution	Acknowledged.

Issue_28	setSwapPausedFor cannot be cancelled early, forcing unnecessary swap downtime
Severity	Informational
Description	The setSwapPausedFor function sets swapPausedUntil to block.timestamp + duration (up to MAX_SWAP_PAUSE of 10 minutes) and provides no mechanism to clear it before the timer expires: > uint40 until = uint40(block.timestamp + duration); > _controls.swapPausedUntil = until; Once the pause is active, _beforeSwap unconditionally reverts all swaps on every pool bound to the hook until block.timestamp passes swapPausedUntil. There is no unpause or cancelPause function. If the owner triggers the halt to protect inventory during an incident (e.g. to safely toggle jitPaused or disableDeferredRemoval) and resolves the issue within seconds, swaps remain bricked for the full chosen duration. The rate-limiting cooldown (SWAP_PAUSE_COOLDOWN) further compounds this: calling setSwapPausedFor again to set a shorter window is blocked until the cooldown expires, so the owner cannot even overwrite the current pause with a shorter one. For a hook that serves as the sole liquidity source for its pools, unnecessary downtime means all trading activity on those pools is halted longer than the incident requires.
Recommendations	Consider adding an early-unpause function that allows the owner to clear swapPausedUntil before the timer expires, without resetting the cooldown clock (so the rate-limit on new pauses remains intact).
Comments / Resolution	Acknowledged.

TransientStorage

The TransientStorage library provides thin EIP-1153 transient-storage helpers (TSTORE / TLOAD) for value types, letting the caller pick an arbitrary storage slot, typically a keccak256 constant, to stash a value that lives only for the current transaction and clears automatically at its end. It is kept type-generic, offering overloaded tstore/tload pairs for bool, uint256, address, and int256 so more per-transaction fields can be added over time without changing the library's shape. It requires an EIP-1153 (Cancun) target, and all reads and writes are marked memory-safe since TSTORE/TLOAD never touch memory.

No issues were identified in this contract.

CurrencySettler

The CurrencySettler library resolves an open currency delta against the Vault, letting a caller settle (pay) a negative delta or take/mint a positive one. settle pays off a debt to the vault: it can burn the caller's ERC-6909 balance, transfer native currency via vault.settle with the value attached, or transfer/transferFrom an ERC-20 to the vault, syncing the vault's currency reserves first in the non-burn cases since deltas are synced before any ERC-20 transfer. It early-returns on a zero amount since some tokens revert when transferred with a zero value, and picks safeTransfer over safeTransferFrom when the payer is the calling contract itself so no self-allowance is required.

No issues were identified in this contract.